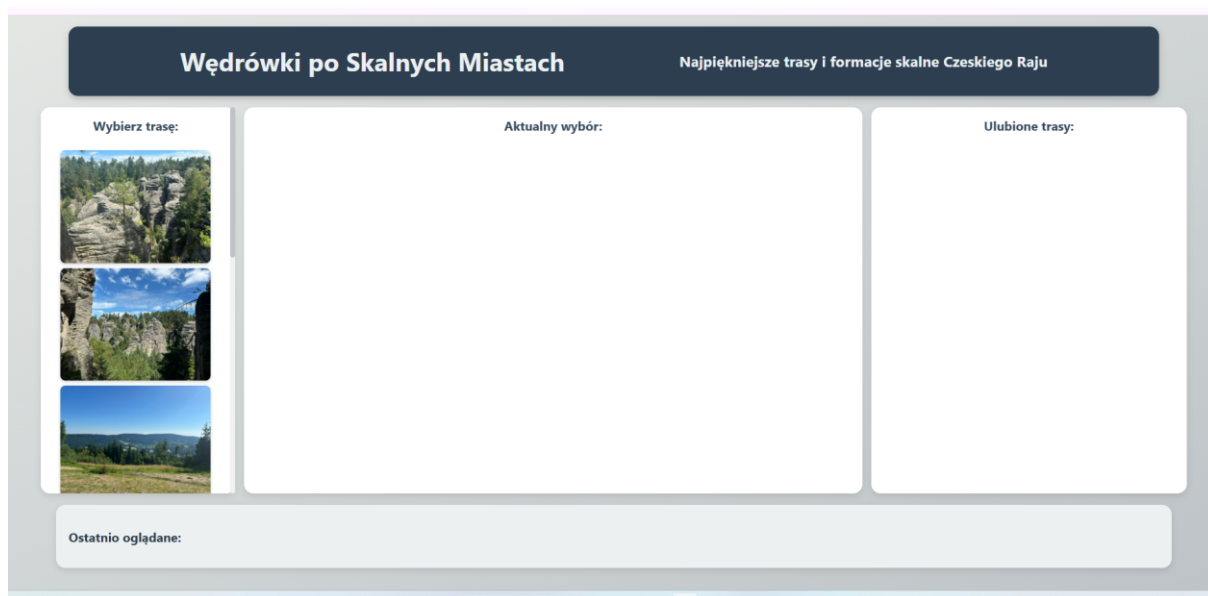


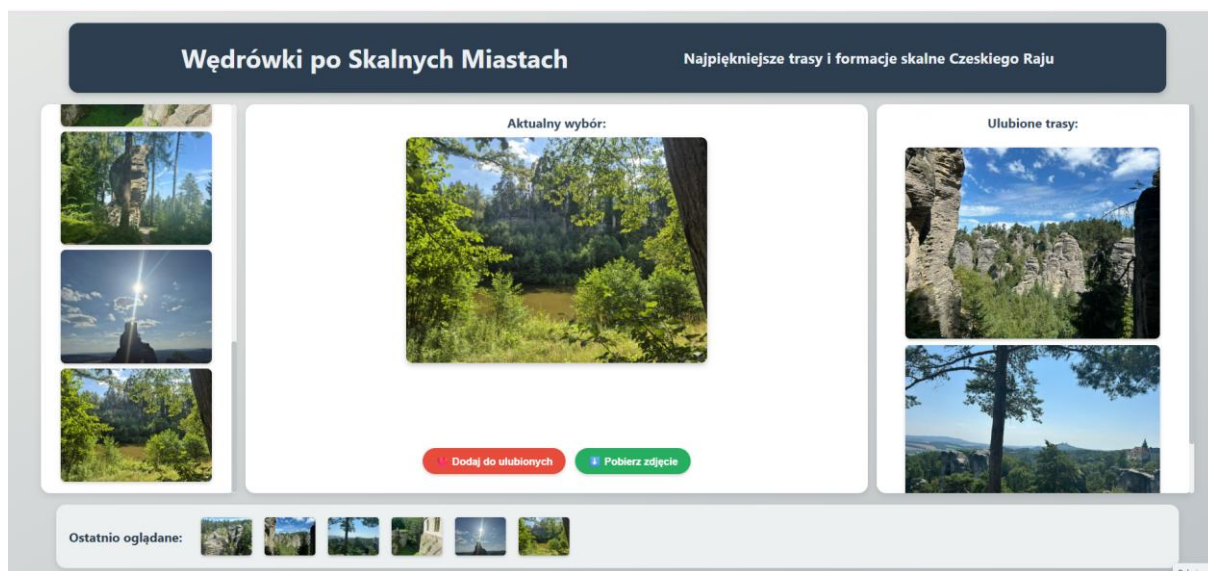
Egzamin poprawkowy z Aplikacji Internetowych

JavaScript – część 1

Aplikacja w stanie początkowym powinna wyglądać następująco:



W trakcie działania, jej wygląd powinien się zmieniać i być uzależniony od wyborów użytkownika:



Zadanie wymaga dopisania funkcji JavaScript, aby uzyskać żadaną funkcjonalność strony. Skrypty muszą działać. W przypadku ograniczonej funkcjonalności działających skryptów można otrzymać za nie część punktów adekwatną do napisanej funkcjonalności i trudności implementacji danej funkcjonalności.

Funkcja *pokaz()*

Wymaga ona dopisania ciała funkcji. Komentarze zawierają teksty na przyciski. Zadaniem tej funkcji jest wstawienie zdjęcia na środkową część ekranu oraz dwóch przycisków.

Przyciski stylizuj za pomocą klas o nazwach:

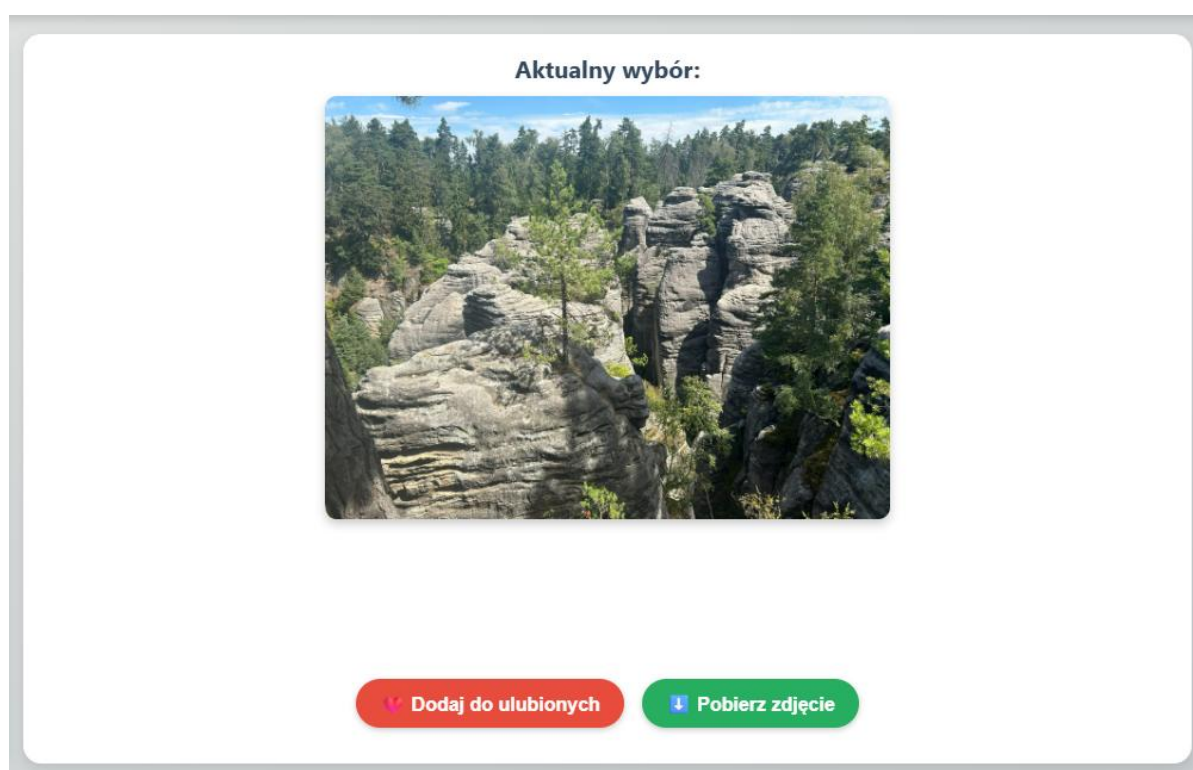
- *btn btn-like*
- *btn btn-download*

Ponadto można dla wygody stworzyć dodatkowy kontener na przyciski i wystylizować go klasą *przyciski-akcji*.

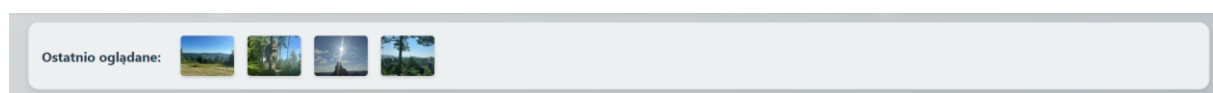
Jeśli wcześniej było inne zdjęcie to należy je usunąć lub podmienić. W oknie *Aktualny wybór*: może znajdować się tylko jedno zdjęcie.

Alternatywnie można obejść tworzenie funkcji z parametrem i stworzyć funkcje *pokaz1()*, *pokaz2()*, itd. Takie rozwiązanie jest jednak mniej punktowne, ale całkowicie dopuszczalne na słabsze oceny. Działająca funkcja uzupełnia środkową część ekranu:

```
function pokaz( numer ){  
    //"❤️ Dodaj do ulubionych"  
    //"⬇️ Pobierz zdjęcie"  
}
```



Dodatkowo funkcja *pokaz(numer)* czy *pokaz1()*, *pokaz2()* powinny uzupełniać historię – *Ostatnio oglądane*:. Znajduje się ona (historia) w stopce (footer) Ta część zadania jest sprawdzana osobno:



Z widocznych przycisków należy oprogramować ❤️ *Dodaj do ulubionych*.

Funkcja *lubie(id)*

Wypełnia ona sekcję o identyfikatorze *prawy* wstawiając tam zdjęcia które użytkownik dodał do ulubionych. Można pominąć sprawdzanie czy zdjęcie zostało już polubione i wtedy może ono pojawiać się kilkukrotnie w prawym oknie. Podobnie jak poprzednio zamiast jednej funkcji z parametrem można napisać kilka funkcji bezparametrowych: *lubie1()*, *lubie2()*, itd. Ale należy pamiętać że jest to rozwiązanie mniej optymalne, a zatem mniej punktowane.

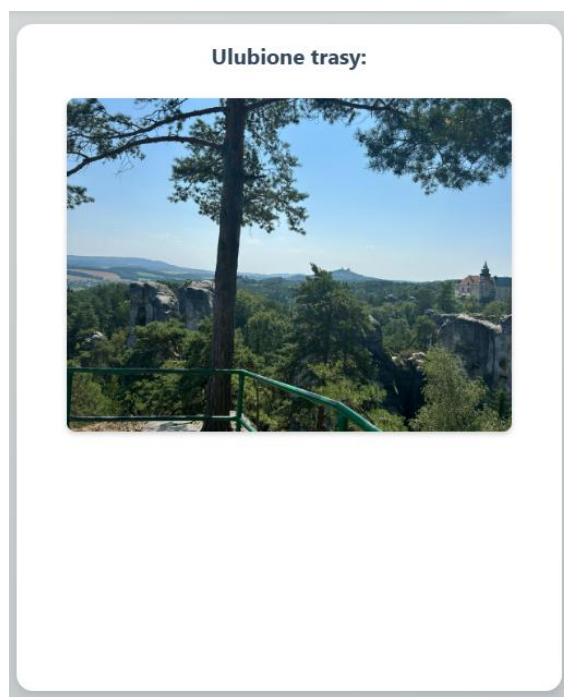


Tabela 1. Wybrane pola i metody modelu DOM języka JavaScript

Wyszukiwanie elementów		Zmiana elementów	
document.getElementById(<i>id</i>)		element.innerHTML = "nowa wartość"	
document.getElementsByTagName(<i>TagName</i>)		element.attribute_name = "nowa wartość"	
document.getElementsByClassName(<i>ClassName</i>)		element.setAttribute(<i>atrybut</i> , <i>wartosc</i>)	
document.getElementsByName(<i>ElementName</i>)		element.style.property_name = "nowa wartość"	
document.querySelector(<i>CSSselector</i>)			
document.querySelectorAll(<i>CSSselector</i>)			
Operacje na elementach dokumentu		Wybrane właściwości obiektu style	
document.createElement(<i>element</i>)		backgroundColor	
document.removeChild(<i>element</i>)		color	
document.appendChild(<i>element</i>)		fontSize	
document.replaceChild(<i>element</i>)		fontStyle = "normal italic oblique initial inherit"	
document.write(<i>text</i>)		fontWeight = "normal lighter bold bolder value initial inherit"	
		listStyleType = "circle decimal disc none square initial..."	

Egzamin poprawkowy z Aplikacji Internetowych

PHP – część 2

Za pomocą narzędzia *phpMyAdmin* wykonaj następujące operacje na bazie danych:

- Utwórz bazę danych o nazwie *czeskiraj*
- Zimportuj bazę z pliku *czeskiraj.sql*.

Aplikacja korzysta z tej bazy danych.

W pełni działająca aplikacja wygląda następująco:

czeskiraj trasy	
id	int(11)
nazwa_trasy	varchar(255)
dlugosc_trasy	decimal(4,2)
trudnosc	int(11)
zdjecie	varchar(50)

Zarządzanie Trasami Turystycznymi

Aplikacja bazodanowa - Czeski Raj

Dodaj nową trasę

Nazwa trasy:
np. Skalne miasto Adršpach

Długość (km):
np. 5.5

Trudność (1-5):

Nazwa pliku zdjęcia:
np. z9.jpg

Dodaj do bazy

Lista dostępnych tras

ID	Nazwa trasy	Długość	Trudność	Miniatura	Plik	Akcje
1	Prachovské skály - Mała pętla	4.50 km	2 / 5		z1.jpg	Usuń
2	Hruboskalsko - Złota Ścieżka	8.20 km	3 / 5		z2.jpg	Usuń
3	Zamek Kost i dolina Pláckének	6.00 km	1 / 5		z3.jpg	Usuń
4	Drábské světničky - Skalné okna	5.50 km	4 / 5		z4.jpg	Usuń

W pliku *index.php* znajduje się skrypt łączenia z bazą, style i formularz. Zadanie polega na napisaniu fragmentów kodu realizującego następujące funkcjonalności:

1. Wstawienie do bazy danych rekordu . Wstawianie odbywa się po wciśnięciu przycisku *Dodaj do bazy*. Wstawianie nie musi obejmować poprawności wpisywanych przez użytkownika danych. Po wstawieniu tabela powinna się automatycznie aktualizować i pokazywać nową trasę. Z oczywistych powodów funkcjonalność ta powinna być napisana w górnym skrypcie.
2. Skrypt wypełniający tabelę jak na dołączonych obrazkach. Usunąć powinno być linkiem wysyłającym żądanie *GET* usunięcia wskazanego rekordu.

ID	Nazwa trasy	Długość	Trudność	Miniatura	Plik	Akcje
1	Prachovské skály - Mała pętla	4.50 km	2 / 5		z1.jpg	Usuń
2	Hruboskalsko - Złota Ścieżka	8.20 km	3 / 5		z2.jpg	Usuń
3	Zamek Kost i dolina Pláckének	6.00 km	1 / 5		z3.jpg	Usuń
4	Drábské světničky - Skalné okna	5.50 km	4 / 5		z4.jpg	Usuń
5	Besedické skály - Skalný Labyrint	4.00 km	3 / 5		z5.jpg	Usuń
6	Szlak pod ruiną zamku Troosky	7.50 km	3 / 5		z6.jpg	Usuń
7	Klášterské skály i zamek Rotštejn	9.00 km	4 / 5		z7.jpg	Usuń
8	Wielka Pętla przez Valdštejn	10.00 km	2 / 5		z8.jpg	Usuń

3. Usuwanie rekordu z bazy danych. Odbywa się po wciśnięciu Usurń w tabeli dostępnych tras. Po usunięciu trasy tabela powinna się automatycznie aktualizować. Z oczywistych powodów funkcjonalność ta powinna być napisana w górnym skrypcie.

Tabela 1. Wybór funkcji języka PHP do obsługi bazy MySQL i MariaDB

Funkcje biblioteki mysqli	Zwracana wartość
<code>mysqli_connect(serwer, użytkownik, hasło, nazwa_bazy)</code>	id połączenia lub FALSE, gdy niepowodzenie
<code>mysqli_select_db(id_polaczenia, nazwa_bazy)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_error(id_polaczenia)</code>	Tekst komunikatu błędu
<code>mysqli_close(id_polaczenia)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_query(id_polaczenia, zapytanie)</code>	Wynik zapytania
<code>mysqli_fetch_row(wynik_zapytania)</code>	Tablica numeryczna odpowiadająca wierszowi zapytania
<code>mysqli_fetch_array(wynik_zapytania)</code>	Tablica zawierająca kolejny wiersz z podanych w wyniku zapytania lub FALSE, jeżeli nie ma więcej wierszy w wyniku zapytania
<code>mysqli_num_rows(wynik_zapytania)</code>	Liczba wierszy w podanym zapytaniu
<code>mysqli_num_fields(wynik_zapytania)</code>	Liczba kolumn w podanym zapytaniu
<code>isset(\$zmienna)</code>	TRUE/FALSE w zależności od tego, czy \$zmienna istnieje

Przykład Przesyłania danych między stronami za pomocą odnośników

Istnieje możliwość przekazywania niedużej liczby parametrów za pomocą zwykłych odnośników dostępnych w języku HTML. W takim przypadku wykorzystuje się metodę GET – konstruuje się odnośnik, na końcu którego należy umieścić znak zapytania a po nim oddzielone ampersandami pary parametr=wartość. Przykład:

strona.html: `<a href="strona.php?imie=Franek&nazwisko=Kowalski">Kliknij tu</a>`

strona.php: `<?php echo "<p>$_GET[imie]</p>"; ?>`